



INSTRUÇÕES PARA A REALIZAÇÃO DO DESAFIO TÉCNICO

Objetivo Geral

Este desafio tem como objetivo avaliar conhecimentos fundamentais de desenvolvimento web, **utilizando Python com Flask**, integração com APIs, banco de dados SQL, Docker, organização de código e tratamento de falhas, por meio da construção de uma aplicação simples de agenda médica. A proposta foi elaborada para permitir que o candidato demonstre raciocínio, autonomia e capacidade de estruturar uma solução compatível com situações encontradas no contexto da empresa.

Parte 1: Construção da Aplicação

1. Descrição do Projeto

Desenvolva uma aplicação web denominada Agenda Médica. A aplicação deverá permitir o login de um usuário e, após a autenticação, exibir os agendamentos médicos disponíveis em uma tabela.

2. Tela de Login

- Criar uma tela de login com os campos usuário ou e-mail e senha.
- Validar as credenciais utilizando os dados armazenados em um banco SQL.
- Exibir uma mensagem clara quando as credenciais estiverem inválidas.
- Após o login válido, redirecionar o usuário para a tela principal da agenda médica.

3. Banco de Dados SQL

- Utilizar SQLite
- Criar, no mínimo, uma tabela de usuários e uma estrutura para armazenar ou registrar os dados necessários à aplicação.
- Disponibilizar um script, migration ou seed para criação do usuário de teste e preparação inicial do banco.

4. Integração com API por Requisições HTTP

- Após a autenticação, a aplicação deverá realizar uma requisição HTTP para buscar os dados dos agendamentos.
- A API poderá ser simulada em um serviço separado dentro do próprio projeto ou disponibilizada por meio de dados mockados em um endpoint HTTP.
- A resposta deverá conter, no mínimo, paciente, CPF, médico, especialidade, data, horário, convênio e status do agendamento.
- A aplicação deverá entregar os dados quando iniciada pelo terminal



Parte 1: Construção da Aplicação - Continuação

5. Exibição da Agenda com Tabulator

Os agendamentos deverão ser apresentados utilizando a biblioteca Tabulator, em formato de tabela.

- Exibir colunas para data, horário, paciente, CPF, médico, especialidade, convênio e status.
- Permitir, no mínimo, busca ou filtro por paciente, CPF ou médico.
- A tabela deverá apresentar uma mensagem adequada quando não houver agendamentos disponíveis.

6. Busca de Dados

A tela principal deverá permitir que o usuário consulte os dados de uma pessoa ou de um agendamento, utilizando CPF, nome ou outro identificador definido pelo candidato.

- Quando houver correspondência, os dados deverão ser exibidos na tabela.
- Quando não houver correspondência, a aplicação deverá informar que nenhum registro foi encontrado.
- Entradas vazias ou inválidas deverão ser tratadas sem provocar erro interno na aplicação.

7. Docker

- Criar um Dockerfile para a aplicação.
- Criar um arquivo docker-compose para iniciar a aplicação e o banco de dados com um único comando.
- Caso seja criada uma API simulada em serviço separado, ela também deverá ser incluída no docker-compose.
- Utilizar variáveis de ambiente para credenciais e configurações sensíveis.

Parte 2: Tratamento de Cenários e Falhas

A aplicação deverá tratar de forma clara e controlada, no mínimo, os seguintes cenários:

- credenciais de login inválidas;
- nenhum agendamento encontrado;
- resposta vazia ou inválida da API;
- indisponibilidade temporária da API;
- erro de conexão com o banco de dados;
- campos obrigatórios ausentes na resposta recebida.



As falhas não devem resultar em páginas quebradas ou mensagens técnicas incompreensíveis para o usuário. Registre logs suficientes para facilitar a identificação da causa do problema.





Parte 3: Entrega e Documentação

1. Repositório

- Disponibilizar todos os arquivos do projeto em um repositório Git.
- Manter os commits organizados e com mensagens compreensíveis.
- Não incluir credenciais reais ou arquivos sensíveis no repositório.

2. README

O arquivo README deverá apresentar:

- descrição breve da solução;
- tecnologias utilizadas;
- instruções para executar o projeto com Docker;
- credenciais do usuário de teste;
- exemplos de uso da aplicação;
- decisões técnicas e eventuais limitações conhecidas.

3. Testes

Crie testes automatizados para, no mínimo, dois comportamentos relevantes da aplicação. Sugestões: login válido e inválido, retorno sem agendamentos, falha da API ou busca de paciente inexistente.

Critérios de Avaliação

A avaliação considerará os seguintes aspectos:

- funcionamento do fluxo de login e consulta da agenda;
- integração correta com a API e com o banco SQL;
- tratamento de erros e mensagens apresentadas ao usuário;
- organização, legibilidade e separação de responsabilidades no código;
- uso adequado do Docker e facilidade para executar o projeto;
- qualidade da documentação e capacidade de explicar as decisões adotadas.

Observações Finais

- Não é necessário desenvolver uma solução excessivamente complexa.
- O candidato deverá estar preparado para apresentar o projeto e explicar suas escolhas técnicas durante a entrevista.
- O uso de ferramentas de inteligência artificial é permitido como apoio, mas o candidato deverá compreender e ser capaz de explicar integralmente o código entregue.



- O projeto deverá ser enviado até a data informada no e-mail de encaminhamento do desafio.
- O link do repositório deverá ser encaminhado ao endereço indicado pelo setor responsável pelo processo seletivo

